

Foundations of Vanishing gradient problem

[Vanishing gradient problem]

1.0 Content Block

Faster hardware Hardware advances have meant that from 1991 to 2015, computer power (especially as delivered by GPUs) has increased around a million-fold, making standard backpropagation feasible for networks several layers deeper than when the vanishing gradient problem was recognized. Schmidhuber notes that this "is basically what is winning many of the image recognition competitions now", but that it "does not really overcome the problem in a fundamental way" since the original models tackling the vanishing gradient problem by Hinton and others were trained in a Xeon processor, not GPUs.

2.0 Content Block

Geometric model Continue using the above one-neuron network, fixing $w = 5$, $x(0) = 0.5$, $u(t) = 0$ $\{\displaystyle w=5, x(0)=0.5, u(t)=0\}$, and consider a loss function defined by $L(x(T)) = (0.855 - x(T))^2$ $\{\displaystyle L(x(T))=(0.855-x(T))^2\}$. This produces a rather pathological loss landscape: as b $\{\displaystyle b\}$ approach -2.5 $\{\displaystyle -2.5\}$ from above, the loss approaches zero, but as soon as b $\{\displaystyle b\}$ crosses -2.5 $\{\displaystyle -2.5\}$, the attractor basin changes, and loss jumps to 0.50. Consequently, attempting to train b $\{\displaystyle b\}$ by gradient descent would "hit a wall in the loss landscape", and cause exploding gradient. A slightly more complex situation is plotted in, Figures 6.

3.0 Content Block

Weight initialization Weight initialization is another approach that has been proposed to reduce the vanishing gradient problem in deep networks. Kumar suggested that the distribution of initial weights should vary according to activation function used and proposed to initialize the weights in networks with the logistic activation function using a Gaussian distribution with a zero mean and a standard deviation of $3.6 / \sqrt{N}$ $\{\displaystyle 3.6/\sqrt{N}\}$, where N $\{\displaystyle N\}$ is the number of neurons in a layer. Recently, Yilmaz and Poli performed a theoretical analysis on how gradients are affected by the mean of the initial weights in deep neural networks using the logistic activation function and found that gradients do not vanish if the mean of the initial weights is set according to the formula: $\max(-1, -8/N)$ $\{\displaystyle \max(-1, -8/N)\}$. This simple strategy allows networks with 10 or 15 hidden layers to be trained very efficiently and effectively using the standard backpropagation.

4.0 Content Block

In machine learning, the vanishing gradient problem is the problem of greatly diverging gradient magnitudes between earlier and later layers encountered when training neural networks with backpropagation. In such methods, neural network weights are updated proportional to their partial derivative of the loss function. As the number of forward propagation steps in a network increases, for instance due to greater network depth, the gradients of earlier weights are calculated with increasingly many multiplications. These multiplications shrink the gradient magnitude. Consequently, the gradients of earlier weights will be exponentially smaller than the gradients of later weights. This difference in gradient magnitude might introduce instability in the training process, slow it, or halt it entirely. For instance, consider the hyperbolic tangent activation function. The gradients of this function are in range $[0,1]$. The product of repeated multiplication with such gradients decreases exponentially. The inverse problem, when weight gradients at earlier layers get exponentially larger, is called the exploding gradient problem. Backpropagation allowed researchers to train supervised deep artificial neural networks from scratch, initially with little success. Hochreiter's diplom thesis of 1991 formally identified the reason for this failure in the "vanishing gradient problem", which not only affects many-layered feedforward networks, but also

recurrent networks. The latter are trained by unfolding them into very deep feedforward networks, where a new layer is created for each time-step of an input sequence processed by the network (the combination of unfolding and backpropagation is termed backpropagation through time).

5.0 Content Block

Since $|\sigma'| \leq 1$, the operator norm of the above multiplication is bounded above by $\|W_{\text{rec}}\|^k$. So if the spectral radius of W_{rec} is $\gamma < 1$, then at large k , the above multiplication has operator norm bounded above by $\gamma^k \rightarrow 0$. This is the prototypical vanishing gradient problem. The effect of a vanishing gradient is that the network cannot learn long-range effects. Recall Equation (loss differential): $\nabla_{\theta} L = \nabla_x L(x_T, u_1, \dots, u_T) [\nabla_{\theta} F(x_{t-1}, u_t, \theta) + \nabla_x F(x_{t-1}, u_t, \theta) \nabla_{\theta} F(x_{t-2}, u_{t-1}, \theta) + \dots]$. The components of $\nabla_{\theta} F(x, u, \theta)$ are just components of $\sigma(x)$ and u , so if $u_t, u_{t-1}, \dots$ are bounded, then $\nabla_{\theta} F(x_{t-k-1}, u_{t-k}, \theta)$ is also bounded by some $M > 0$, and so the terms in $\nabla_{\theta} L$ decay as $M \gamma^k$. This means that, effectively, $\nabla_{\theta} L$ is affected only by the first $O(\gamma^{-1})$ terms in the sum. If $\gamma \geq 1$, the above analysis does not quite work. For the prototypical exploding gradient problem, the next model is clearer.

6.0 Content Block

Recurrent network model A generic recurrent network has hidden states $h_1, h_2, \dots$, inputs $u_1, u_2, \dots$, and outputs $x_1, x_2, \dots$. Let it be parameterized by θ , so that the system evolves as